generates rendered HTML and the client manipulates the existing DOM. When the client wants to update all or part of the page, it makes requests to the server, receives HTML, and inserts that HTML directly into the DOM. This was more or less the original Web 2.0 paradigm and was the interaction supported by early versions of Rails via helper methods that made remote calls and inserted the returned HTML into a DOM element with a given ID.

These days, Hotwire and Turbo allow a more powerful version of this paradigm to create very interactive client experiences while writing little to no JavaScript. Again, it also helps that many client-side flourishes can now be done in CSS.

On the other extreme, you have a single-page JavaScript app that does the maximum amount of work on the client. After sending the original JavaScript, the server is limited to sending data back and forth, probably using JSON, while the client converts that data to DOM elements using templates or JSX or something. The client also manages the state of the application, including the address the browser displays in the address bar, and the way the browser's Back button works. The client is responsible for making sure the server is informed of any data change that needs to be persisted.

Both of these options have their benefits and drawbacks. There's also a middle ground, where individual web pages might have their own rich interactions, but we let the server handle the transition between pages.

Patterns of Web Applications

To make this architecture discussion more concrete, let's look at how these decisions might play out in a specific web app. Slack is a real-time collaboration and chat application that runs in a browser.[24] Later in the book we'll look at how an application might handle real-time chat notification. For now, let's focus on two user interactions: (1) when users click in the sidebar to remain in chat but change the Slack channel they are looking at, and (2) when users click to view their profiles, which completely takes away the chat interface and replaces it with something more like a form.

When the user clicks on a different channel, the basic UI stays in place, but all